

RailRush

IRCTC Clone + Tatkal Simulation Playground

Software Requirements Specification (SRS)

Version 1.0 | Summer Project 2025

Tech Stack: React · Node.js · PostgreSQL · Redis · C++

1. Introduction

1.1 Project Overview

RailRush is a summer learning project that replicates the core booking experience of IRCTC (Indian Railway Catering and Tourism Corporation) while introducing a unique Tatkal Simulation Playground — an interactive feature where users can experience and study high-concurrency ticket booking in a controlled environment.

The primary goal is NOT to clone every IRCTC feature. The goal is to understand and demonstrate: concurrent systems, race conditions, lock-free programming, caching strategies, and real-time communication — all wrapped inside a relatable, real-world product.

1.2 Why This Project?

- IRCTC's Tatkal window is one of the hardest real-world concurrency problems in India
- 10,000+ users competing for ~100 seats in under 10 seconds
- Every concept learned here maps directly to SWE interview questions at top companies
- The simulation playground is a unique feature no standard IRCTC clone has

1.3 Project Codename

Codename: RailRush. This name will be used throughout the codebase, README, and documentation.

1.4 Scope — What We Are Building

Included in scope:

- User auth (register, login, JWT sessions)
- Train search (by source, destination, date)
- Seat availability display
- Normal ticket booking flow
- Tatkal booking flow (via C++ engine)
- Tatkal Simulation Playground (the star feature)
- Real-time seat updates via WebSocket
- Basic booking history page

Explicitly OUT of scope (to keep timeline realistic):

- Payment gateway integration (mock payment only)
- PNR status tracking
- Cancellation and refund flows
- Admin panel
- Mobile app
- Email/SMS notifications

2. System Architecture

2.1 High-Level Architecture

The system has 5 major layers. Each layer has a single responsibility — this is the most important design principle in the project.

Layer	Technology	Responsibility	Why This Choice
Frontend	React + Vite	User interface, simulation playground UI	Fast dev server, component-based, easy state management
API Gateway	Node.js + Express	Route requests, auth, rate limiting	JavaScript everywhere, huge ecosystem, easy to learn
Tatkal Engine	C++ (compiled binary)	Lock-free seat reservation under high load	True multi-threading, atomic ops, nanosecond control
Cache Layer	Redis	Seat bitmap, booking queue, session store	In-memory speed, built-in data structures like BITSET
Database	PostgreSQL	Permanent storage of users, bookings, trains	ACID transactions, row-level locking, reliable

2.2 Communication Between Layers

- React → Node.js: REST API (JSON over HTTP) for normal flows
- React → Node.js: WebSocket (Socket.io) for real-time simulation updates
- Node.js → C++ Engine: gRPC (a fast protocol used in real industry systems — explained below)
- Node.js → Redis: ioredis client library
- Node.js → PostgreSQL: pg (node-postgres) library

Why gRPC between Node.js and C++?

gRPC is like REST but faster and strongly typed. You define the data structure once in a .proto file and both sides (Node and C++) auto-generate code from it. This is how Google, Netflix, and Uber connect services written in different languages. For this project it is the standard way to call C++ from Node.js — using `child_process` or a raw socket would be messy and error-prone.

2.3 Folder Structure

Folder	Purpose
/client	React frontend (Vite project)
/server	Node.js API (Express app)
/engine	C++ Tatkal engine source code
/proto	gRPC .proto files shared by Node and C++
/db	SQL migration files and seed data
/docs	This SRS and other documentation

Folder	Purpose
docker-compose.yml	Spins up Postgres + Redis locally with one command

3. Database Design

3.1 Tables Overview

PostgreSQL will have 5 core tables. The schema is kept deliberately simple — no over-engineering.

Table: users

Column	Type	Description
id	UUID (PK)	Unique user identifier
name	VARCHAR(100)	Full name
email	VARCHAR(150) UNIQUE	Login email
password_hash	TEXT	Bcrypt hashed password — never store plain text
created_at	TIMESTAMP	Account creation time

Table: trains

Column	Type	Description
id	SERIAL (PK)	Train number
name	VARCHAR(100)	Train name (e.g. Chennai Express)
source	VARCHAR(50)	Departure station
destination	VARCHAR(50)	Arrival station
departure_time	TIME	Scheduled departure
total_seats	INTEGER	Total seats on this train

Table: schedules

Column	Type	Description
id	SERIAL (PK)	Schedule ID
train_id	INTEGER (FK → trains)	Which train
travel_date	DATE	The specific date of travel
available_seats	INTEGER	Seats remaining (syncd from Redis)
tatkal_available	INTEGER	Tatkal seats remaining

Table: bookings

Column	Type	Description
id	UUID (PK)	Booking reference number
user_id	UUID (FK → users)	Who booked

Column	Type	Description
schedule_id	INTEGER (FK → schedules)	Which train on which date
seat_number	INTEGER	Assigned seat
booking_type	ENUM('normal','tatkai')	Type of booking
status	ENUM('confirmed','failed')	Result of booking attempt
booked_at	TIMESTAMP	When the booking was made

Table: simulation_runs

This table is unique to RailRush — no real IRCTC has this.

Column	Type	Description
id	UUID (PK)	Simulation run ID
user_id	UUID (FK → users)	Who ran the simulation
bot_count	INTEGER	How many bots competed
total_seats	INTEGER	Seats available in the simulation
user_rank	INTEGER	Where the user placed (1 = got first seat)
user_latency_ms	INTEGER	User's booking request latency
p50_latency_ms	INTEGER	Median latency across all requests
p99_latency_ms	INTEGER	99th percentile latency
throughput_rps	INTEGER	Requests processed per second
ran_at	TIMESTAMP	When simulation ran

4. API Design (REST Endpoints)

4.1 Auth Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Create new user account	No
POST	/api/auth/login	Login, receive JWT token	No
POST	/api/auth/logout	Invalidate session	Yes

4.2 Train & Search Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/trains/search?from=&to=&date=	Search trains by route and date	No
GET	/api/trains/:id/seats?date=	Get seat availability for a train	No

4.3 Booking Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/bookings/normal	Book a regular ticket	Yes
POST	/api/bookings/tatkal	Book a Tatkal ticket (goes to C++ engine)	Yes
GET	/api/bookings/my	Get user's booking history	Yes

4.4 Simulation Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/simulation/start	Start a simulation run (bot_count, seat_count in body)	Yes
GET	/api/simulation/results/:id	Fetch results of a completed simulation	Yes
GET	/api/simulation/history	Get user's past simulation runs	Yes

4.5 WebSocket Events (Socket.io)

Event	Direction	Description
simulation:start	Client → Server	User signals they're ready to begin
simulation:tick	Server → Client	Live update: queue position, seats remaining, last claim
simulation:end	Server → Client	Final results: rank, latency stats, throughput

Event	Direction	Description
seat:claimed	Server → Client	A seat was just claimed (normal booking, real-time UI update)

5. C++ Tatkal Engine (The Core of This Project)

5.1 What It Does

The C++ engine is a standalone process that handles ONLY one thing: accepting concurrent seat reservation requests and granting them fairly and safely with zero data corruption.

This is the part that teaches you the real systems programming concepts. Read every line of this section carefully.

5.2 The Problem It Solves

Imagine 10,000 users all click 'Book' at the same time. Without proper concurrency control:

- Two users might both be told 'Seat 42 is available'
- Both write to the DB at the same time
- Both get confirmed — but only one seat exists
- This is called a Race Condition — the #1 concurrency bug

The C++ engine prevents this using atomic operations (explained below).

5.3 Key Concepts Used (Plain English)

Concept	What It Means	Where Used
Atomic CAS	Compare-And-Swap: check a value and change it in ONE uninterruptible step	Claiming a seat bit
Thread Pool	A fixed set of pre-created threads that pick up work from a queue	Handling 10k concurrent requests
Lock-free Queue	A queue where threads don't wait for each other — they use atomic ops instead	Incoming request queue
Seat Bitmap	Each bit = one seat. Bit 0 = available, Bit 1 = taken. 100 seats = 100 bits	Fast availability check
gRPC Server	The engine listens on a port. Node.js calls it like a function call	Node.js ↔ C++ bridge

5.4 Engine Flow (Step by Step)

1. Node.js sends a 'reserve seat' gRPC call with { userId, scheduleId }
2. Engine picks up the request from its thread pool queue
3. Engine does atomic CAS on the seat bitmap:
 - Read current bitmap value
 - Find first 0 bit (available seat)
 - Try to flip it to 1 using compare_exchange_strong
 - If another thread beat us to it, retry — this is called a spin loop
4. If a seat was claimed: return { success: true, seatNumber: X }
5. If no seats remain: return { success: false, reason: 'SOLD_OUT' }

6. Node.js writes the confirmed booking to PostgreSQL

5.5 Why Not Just Use a Mutex (Lock)?

A mutex (lock) would work but it serializes everything — only one thread runs at a time. With 10,000 requests this creates a massive bottleneck. Atomic CAS lets ALL threads try simultaneously and only ONE wins per seat — no waiting, no bottleneck. This is called lock-free programming and is used in production systems at Google, Meta, and Amazon.

5.6 Files in /engine

File	Purpose
main.cpp	Entry point — starts the gRPC server and thread pool
seat_manager.h / .cpp	The atomic bitmap logic — this is the most important file
request_queue.h / .cpp	Lock-free queue for incoming requests
tatkal.proto	Shared with /proto — defines the gRPC interface
CMakeLists.txt	Build configuration (how to compile the C++ code)

6. Tatkal Simulation Playground

6.1 Feature Overview

This is the unique feature of RailRush that no other IRCTC clone has. A user can simulate the Tatkal booking rush — set the number of competing bots, configure the window, and compete against them in real time.

6.2 User Flow

Step	What the User Sees
1. Config screen	Sliders for: number of bots (100 to 10,000), number of seats (10 to 200), window duration (5s to 30s), network jitter (0ms to 200ms)
2. Countdown	3-2-1 countdown. User prepares to click 'Book' the moment window opens
3. Live race	Real-time feed: 'Bot #3421 claimed Seat 7', queue position counter, seats remaining bar
4. User books	User clicks 'Book' button — their request enters the C++ engine alongside bots
5. Result screen	Your rank: 47 / 10,000 — You got a seat! Latency histogram, throughput graph

6.3 How Bots Are Simulated

The Node.js Simulation Controller spawns N async tasks (using Promise.all). Each task:

- Waits for a random jitter (0 to maxJitter ms) to simulate real network variance
- Calls the C++ engine via gRPC with a fake userId
- Records the latency and result

Note: For up to ~1,000 bots, Node.js async is sufficient. For 10,000 bots, the C++ engine itself spawns the bot threads internally to avoid overloading the Node.js event loop. The agent will explain this clearly in code comments.

6.4 Result Metrics Explained

Metric	What It Means	Why It Matters
Your Rank	Position of your request in the queue (1 = fastest)	Shows where you'd stand in a real Tatkal rush
p50 Latency	50% of requests were faster than this	The 'typical' user experience
p99 Latency	99% of requests were faster than this	Shows worst-case slowness — important for SLA design
Throughput (RPS)	How many requests the engine processed per second	Measures raw system capacity
Queue drain time	Total time to process all requests	End-to-end window duration

7. Frontend (React)

7.1 Pages

Page / Route	Description
/	Landing page — explains RailRush, link to register/login
/search	Train search form and results list
/train/:id	Seat map and booking form for a specific train
/booking/confirm	Booking confirmation screen
/my-bookings	User's booking history
/simulation	Tatkal Simulation Playground (the star page)
/simulation/results/:id	Results of a completed simulation run
/login	Login form
/register	Register form

7.2 State Management

Use React Context + useReducer for global state (auth, current user). Keep it simple — no Redux. Local component state (useState) for forms and UI interactions. This is the right level of complexity for an intermediate-level project.

7.3 Key Libraries

Library	Purpose	Install Command
React Router v6	Page routing and navigation	npm install react-router-dom
Axios	HTTP requests to the API	npm install axios
Socket.io-client	WebSocket connection for real-time updates	npm install socket.io-client
Recharts	Latency histograms and throughput charts in simulation results	npm install recharts
Tailwind CSS	Utility-first styling — fast and clean	npm install -D tailwindcss

8. 6-Week Development Plan

8.1 Timeline Overview

Week	Theme	Deliverable by End of Week
Week 1	Foundation	Project scaffolded, DB running, auth working, trains seeded
Week 2	Core Booking	Train search, seat availability, normal booking flow end-to-end
Week 3	C++ Engine	C++ Tatkall engine compiled, gRPC bridge working, Tatkall booking live
Week 4	Simulation (Backend)	Simulation controller, bot spawning, result recording working
Week 5	Simulation (Frontend)	Full playground UI, real-time WebSocket feed, results charts
Week 6	Polish & Demo	README, seed data, demo video, chaos mode (bonus), bug fixes

8.2 Week 1 — Foundation

- Set up monorepo: /client, /server, /engine, /db folders
- docker-compose.yml for Postgres + Redis (one command to start everything)
- Run DB migrations: create all 5 tables
- Seed train data (at least 10 trains, 30 schedules)
- Node.js: POST /api/auth/register and POST /api/auth/login with JWT
- React: Login and Register pages connected to API
- Test: register a user, login, get JWT — done

8.3 Week 2 — Core Booking

- Node.js: GET /api/trains/search endpoint
- Node.js: POST /api/bookings/normal with Postgres transaction
- Redis: cache seat availability (5 min TTL, invalidate on booking)
- React: Train search page, seat display, booking form
- React: My Bookings page
- Test: search a train, book a seat, see it in history — done

8.4 Week 3 — C++ Engine

- Write seat_manager.cpp with atomic bitmap
- Write gRPC server in C++ using tatkall.proto
- Write gRPC client in Node.js
- Connect POST /api/bookings/tatkall to C++ engine
- Test: send 100 concurrent requests, verify no double-booking
- Add comments in code explaining every atomic operation

8.5 Week 4 — Simulation Backend

- Write Simulation Controller in Node.js

- Bot spawning logic (async for small counts, C++ threads for large)
- POST /api/simulation/start → returns simulation run ID
- Record all metrics to simulation_runs table
- Socket.io: emit simulation:tick events during run
- Test: run 1000-bot simulation, verify metrics saved correctly

8.6 Week 5 — Simulation Frontend

- Config screen with sliders (bot count, seats, jitter, duration)
- Live race view — WebSocket feed showing real-time claims
- User 'Book' button during simulation window
- Results screen with Recharts: latency histogram, throughput bar, rank display
- Simulation history page

8.7 Week 6 — Polish

- Write complete README with architecture diagram and setup instructions
- Add error handling everywhere (currently most paths have none)
- Add chaos mode: randomly kill a worker thread, observe recovery (bonus feature)
- Record a 3-minute demo video showing the simulation
- Deploy to Railway or Render (optional but impressive)

9. Agent Prompting Strategy

9.1 The Golden Rule

Every prompt to your AI agent must include three things: (1) what file to create, (2) what the code must do, and (3) explicit instructions on how comments should be written. Do not skip the comments instruction — it is what makes the code understandable.

9.2 Comment Style Rule for Every Prompt

Always include this sentence in your prompts:

"Write comments like a senior explaining to a junior. Before every non-obvious line, add a comment explaining WHY not just what. If you use any advanced technique (lock-free, atomic, gRPC, thread pool), add a paragraph comment explaining what it is and why we chose it over the simpler alternative."

9.3 Prompt Template

Use this template for every agent session:

"Project: RailRush (IRCTC Clone + Tatkai Simulation). Tech: React, Node.js, PostgreSQL, Redis, C++. I am an intermediate developer who needs to understand every line. Task: [YOUR SPECIFIC TASK]. File to create/edit: [FILE PATH]. Requirements: [BULLET LIST]. Comments rule: explain every non-obvious line and every advanced concept. Keep functions short (under 30 lines each). No external libraries except what's listed in the SRS."

9.4 Week-by-Week Prompt Topics

Week	Prompt Topic	Key Phrase to Include in Prompt
1	docker-compose.yml for Postgres + Redis	'explain each service block and why we need both'
1	DB migration SQL for all 5 tables	'comment each column with why this type was chosen'
1	JWT auth in Node.js (register + login)	'explain what JWT is and why we use it instead of sessions'
2	Train search with Redis caching	'explain the cache-aside pattern in comments'
2	Normal booking with Postgres transaction	'explain what BEGIN / COMMIT / ROLLBACK does and why'
3	C++ seat_manager.cpp with atomic bitmap	'explain compare_exchange_strong line by line'
3	gRPC .proto file + Node.js client	'explain what gRPC is vs REST in a top comment'
4	Simulation controller Node.js	'explain why Promise.all is used and its limits'
5	React simulation playground UI	'explain the useReducer pattern used for simulation state'
5	Socket.io real-time feed component	'explain the useEffect cleanup for socket disconnect'
6	Chaos mode: random thread kill	'explain what happens when a thread dies mid-request'

10. Error Handling Strategy

This is something most students skip and regret. Define it upfront.

Scenario	What Should Happen	HTTP Code
User tries to book a sold-out train	Return clear error: 'No seats available'	409 Conflict
Two users claim the same seat simultaneously	C++ engine grants to first, second gets 'Seat taken, retrying...'	409 Conflict
Invalid JWT token	Return 'Unauthorized', clear cookie	401
DB connection fails	Return 'Service temporarily unavailable', log error	503
Redis unavailable	Fall through to DB (slower but functional)	200 (degraded)
C++ engine process crashes	Node.js catches gRPC error, returns 503, logs for restart	503
Simulation bot count exceeds 10,000	Validate on server, reject with clear message	400

11. What You Will Learn (Learning Map)

Concept	Where You Learn It	Industry Relevance
Race conditions	Week 3 — see what breaks without atomics	Asked in every systems interview
Atomic CAS operations	C++ seat_manager.cpp	Used in Java, C++, Go standard libraries
Lock-free programming	C++ engine thread pool	High-frequency trading, game engines, OS kernels
Cache-aside pattern	Redis seat caching in Week 2	Used at every major tech company
Optimistic vs pessimistic locking	Compare Postgres FOR UPDATE vs atomic CAS	Database interview staple
WebSocket real-time comms	Socket.io simulation feed	Slack, Zerodha, Google Docs all use this
gRPC polyglot services	Node.js ↔ C++ bridge	Standard at Google, Uber, Lyft
p50/p99 latency metrics	Simulation results dashboard	SRE and backend interviews everywhere
Connection pooling	Postgres pg pool config	Prevents DB from being overwhelmed
JWT auth flow	Week 1 auth endpoints	Standard in every web product

End of SRS — RailRush v1.0

Build it. Break it. Understand it.